Hands-On Pertemuan 3 - Framework Security Update Lab

Mata kuliah TI215 Keamanan Aplikasi Web - Institut Teknologi Tangerang Selatan, 2026/2027 Ganjil

Nama	Farel Eka Saputra	NIM	1002250027
Prodi	Teknologi Informasi	Tanggal praktikum	14 Oktober 2026

Nama	Fauzan atillah	NIM	1002250039
Prodi	Teknologi Informasi	Tanggal praktikum	14 Oktober 2026

Nama	Hafiz Hablillah	NIM	1003250012
Prodi	Teknologi Informasi	Tanggal praktikum	14 Oktober 2026

Lingkungan: Windows 11 (Command Prompt dan PowerShell), Node.js v26.11.1. Seluruh pengujian hanya pada http://localhost:3000 (aplikasi lokal dari ZIP starter lab); tidak ada target pihak ketiga. Pada PowerShell digunakan curl.exe

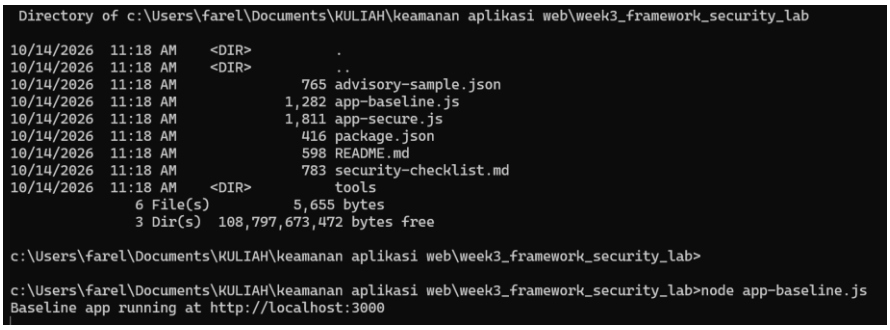
Ringkasan Temuan

Aplikasi baseline berjalan tanpa satu pun dari empat header keamanan yang dicek (header_check.js: 4 dari 4 MISSING) dan menyisipkan parameter q ke HTML tanpa escaping; payload <script>alert(1)</script> benar-benar dieksekusi browser (popup "localhost:3000 says 1"), sehingga terbukti reflected XSS.

Versi secure menampilkan keempat header (4 dari 4 OK) beserta Permissions-Policy, dan memakai escapeHtml() sehingga input diperlakukan sebagai data. Review kode menemukan kelemahan yang masih tersisa pada versi secure: respons 404 tidak memakai securityHeaders() dan fungsi page() memasukkan body tanpa escape. Pada sisi dependency, npm audit melaporkan lodash <=4.17.23 berstatus critical (7 advisory); lodash belum di-import oleh kode sehingga belum reachable, tetapi tetap perlu diperbaiki.

1. Evidence Baseline

Perintah: node -v, cd week3_framework_security_lab, node app-baseline.js; di terminal baru curl.exe -i http://localhost:3000/ dan node tools/header_check.js http://localhost:3000/.



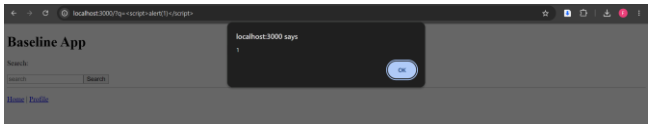
Gambar 1 (Evidence 0). Folder lab dan baseline app berjalan di http://localhost:3000.



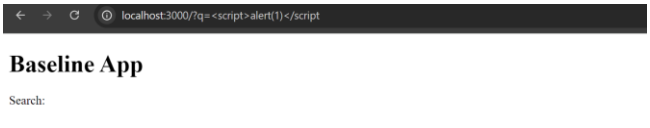
Gambar 2 (Evidence 1). curl.exe -i tanpa header keamanan; header_check.js: CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy semuanya MISSING.

2. Rendering Data User (Evidence 2)

URL uji: http://localhost:3000/?q=<script>alert(1)</script>. Pada baseline script dieksekusi (Gambar 3) karena input dianggap kode, bukan data. Percobaan awal dengan penutup tag tidak lengkap (</script tanpa >) tidak memunculkan alert, tetapi form dan link Home/Profile hilang karena sisa HTML terbaca sebagai isi script (HTML injection, Gambar 4).



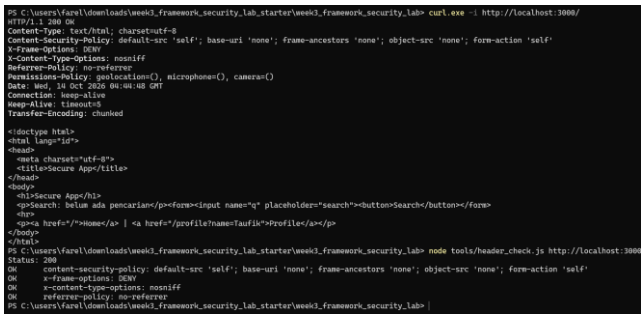
Gambar 3. Baseline: JavaScript dieksekusi (alert "1").



Gambar 4. Penutup tag tidak lengkap: struktur halaman rusak.

3. Evidence Secure dan Perbandingan

Setelah node app-secure.js, curl.exe -i memuat CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer, dan Permissions-Policy; header_check.js menunjukkan 4 baris OK (Gambar 5). Payload yang sama (curl.exe -s -G http://localhost:3000/ --data-urlencode "q=<script>alert(1)</script>") menghasilkan teks literal yang tidak dieksekusi: Search: <script>alert(1)</script>



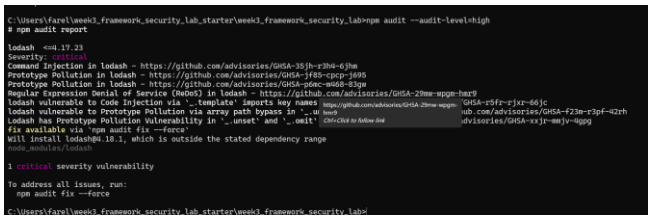
Gambar 5. Versi secure: header lengkap dan header_check.js 4 baris OK.

Aspek	Baseline	Secure
Secure headers	0 dari 4 MISSING (Gambar 2)	4 dari 4 OK + Permissions-Policy; respons 404 belum
Output escaping	Tidak ada; <script> dieksekusi	escapeHtml(): < menjadi <
CSP	Tidak ada	default-src 'self'; base-uri 'none'; frame-ancestors 'none'; object-src 'none'; form-action 'self'
Frame protection	Tidak ada	X-Frame-Options: DENY
Referrer policy	Tidak ada	no-referrer

4. Checklist SAST Manual (Evidence 3)

File	Source	Sink	Kontrol ada	Kontrol hilang	Rekomendasi
app-baseline.js	url.searchParams.get('q') (baris 24); get('name') (baris 34)	res.end(page(..., Search: ${q})) (baris 29); Nama: ${name} (baris 36)	Tidak ada	Output escaping, CSP, X-Frame-Options, nosniff, Referrer-Policy	escapeHtml() / template auto-escape, CSP, middleware secure headers (helmet)
app-secure.js	q dan name (sama)	res.end(page(...)); ${body} mentah di page() (baris 35)	escapeHtml() (baris 6-13, 48, 55); securityHeaders() + CSP (baris 15-24)	Header pada respons 404 (baris 59); page() tidak mengescape body (bergantung disiplin pemanggil); escape hanya valid untuk konteks teks HTML	Header untuk semua respons, template engine auto-escape, test regresi

5. Dependency Triage (SCA)



Gambar 6. npm install (70 package) lalu npm audit --audit-level=high: lodash <=4.17.23 critical, 7 advisory; fix lodash@4.18.1.

Package	Versi	Severity	Impact / Fix / Prioritas
lodash	4.17.11 (ter-pin eksak)	critical (npm audit); high (advisory-sample.json)	Command/code injection via _.template, prototype pollution (_.unset/_.omit), ReDoS (GHSA-35jh-r3h4-6jhm, GHSA-jf85-cpcp-j695, GHSA-r5fr-rjxr-66jc, GHSA-29mw-wpgm-hmr9). Fix: upgrade ke 4.18.1 atau hapus dependency. Prioritas: P2 (sprint ini).

Alasan prioritas: severity critical dan patch tersedia, tetapi app-baseline.js, app-secure.js, dan header_check.js hanya memakai modul bawaan Node (http, url) dan tidak meng-import lodash, sehingga fungsi rentan belum reachable. Karena itu bukan P1; namun dependency tidak terpakai tetap menambah permukaan serangan. Risiko breaking change rendah (tidak ada kode pemanggil); versi ter-pin eksak sehingga package.json perlu diubah. Validasi: npm audit --audit-level=high tanpa temuan dan test regresi lulus.

6. Rekomendasi Teknis

#	Masalah dan bukti	Tindakan	Validasi
1	Respons 404 pada app-secure.js tidak memakai securityHeaders() (SAST baris 59); header hanya terbukti pada route / (Gambar 5).	Terapkan header di satu titik untuk semua respons, atau pakai helmet (sudah ada di package.json tetapi tidak dipakai).	header_check.js pada /, /profile, dan /nope: semua OK.
2	page() memasukkan ${body} mentah (baris 35); endpoint baru yang lupa escapeHtml() langsung rentan XSS (dampak: Gambar 3).	Gunakan template engine dengan auto-escape (EJS, Pug, Nunjucks) atau tagged-template yang mengescape semua interpolasi.	Uji payload <script> dan <img onerror> di setiap endpoint; hasil harus <. Simpan sebagai test otomatis.
3	lodash@4.17.11 critical (Gambar 6), tidak dipakai kode.	Hapus dependency, atau upgrade ke versi aman dan longgarkan pin menjadi range (^).	npm audit --audit-level=high exit code 0; npm ci dan test lulus.
4	header_check.js selalu exit code 0 walau header MISSING (process.exit(1) hanya pada error koneksi), sehingga tidak bisa jadi gerbang CI.	Ubah agar process.exit(1) bila ada header MISSING; jalankan pada pipeline.	Baseline: exit code 1; secure: exit code 0.

7. Refleksi

Dibantu framework/middleware secara otomatis: auto-escaping pada template, header keamanan standar (mis. helmet, Spring Security, Django SecurityMiddleware), proteksi CSRF/session, serta pemberitahuan advisory dependency (npm audit, Dependabot).

Masih butuh judgment developer: memilih konteks escaping yang benar (HTML, atribut, JavaScript, URL), merancang CSP yang tidak merusak fitur, menilai reachability dan prioritas advisory (severity saja tidak cukup, seperti pada lodash), memastikan semua jalur respons termasuk error terlindungi, serta authorization